

COCOMO II (Post-Architecture) aplicado a Mi Ruta

Estimación de esfuerzo, cronograma, personal y presupuesto sobre datos reales del proyecto

Carrera	Sistemas Informáticos	Docente	Ing. Carolina Aguilar
Asignatura	Gestión y Mejoramiento de la Calidad de Software	Turno	Noche
Proyecto	Mi Ruta — planificación y pago de transporte público (Cochabamba, Bolivia)		
Fecha de entrega	09/07/2026		

Nota metodológica: este ejercicio se resuelve sobre un proyecto que ya tiene código real y un tracker de seguimiento real (docs/AsyncDaily/Revision QA.xlsx, docs/INFORME_AVANCE_SPRINT.md). Eso permite **medir** el tamaño en vez de adivinarlo y, en la Parte 5, comparar la predicción del modelo contra lo que el equipo realmente hizo. Los valores marcados **(medido)** vienen del repositorio; los marcados **(aproximado)** son estimaciones razonables donde no existe un dato exacto.

PARTE 1

Tamaño del proyecto en KSLOC

En vez de estimar por juicio experto "a ciegas", se **midió directamente** el código fuente (lib/**/*.dart, líneas no vacías y sin contar líneas que son solo comentario //), descompuesto por módulo — el mismo criterio de descomposición que pide el ejercicio, aplicado sobre datos reales.

MÓDULO (CLEAN ARCHITECTURE)	SLOC MEDIDO
features/user — mapa, wallet, QR, beneficios, notificaciones, historial	18,762
features/driver — vehículo, ingresos, rutas, operaciones	4,906
features/admin — usuarios, rutas, privilegios, reportes	4,451
features/routes — GTFS, planificador multi-tramo, SQLite	2,454
features/tickeador	1,751
features/auth	1,655

MÓDULO (CLEAN ARCHITECTURE)	SLOC MEDIDO
core/ — DI, ruteo, tema, utilidades	1,340
features/stops	517
features/presidente	415
services/ (raíz) + main.dart	~187
features/payment, features/shared — stubs sin implementar	0
Total medido (lib/)	36,822

Tamaño usado en el cálculo: Size = 37 KSLOC (36,822 redondeado). No se incluyen tests ni archivos generados por plugins, siguiendo el criterio estándar de COCOMO de contar solo código fuente entregado.

PARTE 2

Factores de escala y exponente E

Los 5 factores de escala se calificaron con criterio específico al proyecto real — cada uno respaldado por evidencia del propio historial del equipo (`git log --merges`, `docs/INFORME_AVANCE_SPRINT.md`).

FACTOR	NIVEL	SFJ	JUSTIFICACIÓN ESPECÍFICA AL PROYECTO
PREC — Precedentedness	Bajo	4.96	Primera vez que este equipo construye este producto exacto (transporte + GTFS + pagos + 5 roles). Prueba directa: dos personas reconstruyeron el mismo módulo (Tickeador) sin saber que el otro ya lo había hecho.
FLEX — Development Flexibility	Alto	2.03	Sin gobernanza rígida de requisitos: el mismo ID (RQ-65, RQ-78, RQ-80) terminó significando cosas distintas en el backlog "oficial" vs. lo que cada dev construyó, sin proceso de aprobación previo.
RESL — Architecture/Risk Resolution	Bajo	5.65	Riesgos centrales sin resolver hasta el final: 3 tablas de ruteo por rol inconsistentes, reglas de Firestore completamente abiertas hasta los últimos commits, superadmin duplicado con dos listas de emails distintas.
TEAM — Team Cohesion	Bajo	4.38	27 personas registradas, solo 19 reportaron un daily y 4 nunca lo hicieron — incluida la autora de uno de los dos paneles de Presidente duplicados.
PMAT — Process Maturity	Bajo	6.24	Hay tracker, dailies y líderes, pero sin aplicación consistente: ~25 ramas fusionadas a mano sin rama de integración ni gate de revisión; un archivo con credenciales reales llegó a commitarse.

Cálculo de E

$$\Sigma SF_j = 4.96 + 2.03 + 5.65 + 4.38 + 6.24 = 23.26$$
$$E = B + 0.01 \times \Sigma SF_j = 0.91 + 0.01 \times 23.26 = 0.91 + 0.2326 = 1.1426 \approx 1.14$$

Un E de 1.14 (por encima de 1.0) indica **deseconomías de escala**: al crecer el tamaño del sistema, el esfuerzo crece más que proporcionalmente — consistente con un proyecto donde duplicar personas no duplicó avance: se duplicó, literalmente, el trabajo de Tickeador y de Presidente en vez de sumarse.

PARTE 3

Multiplicadores de esfuerzo (EM)

Se eligieron **9 de los 17 drivers** (más de los 6 mínimos pedidos); el resto se deja en Nominal (= 1.00, no afecta el producto).

DRIVER	NIVEL	EM	JUSTIFICACIÓN ESPECÍFICA AL PROYECTO
RELY — Confiabilidad requerida	Alto	1.10	Maneja dinero real (saldo, cobro por QR, recargas). Una falla causa pérdida financiera directa, no de vidas.
CPLX — Complejidad del producto	Alto	1.17	Planificador multi-tramo (hasta 3 tramos bus + caminata, deduplicado por ref directionId), ingestión GTFS, navegación en tiempo real, 5 roles distintos.
DATA — Tamaño de base de datos	Alto	1.14	Firestore con 10+ colecciones activas más una caché SQLite local espejada (rutas, config).
PCON — Continuidad del personal	Bajo	1.12	Un solo dev concentra el 18% de los dailies mientras 4 colaboradores activos nunca reportaron ninguno.
ACAP — Capacidad de analistas	Bajo	1.19	Equipo de estudiantes en su primer proyecto de este tamaño; se revirtieron decisiones de diseño (dos paneles de Presidente, dos Tickeador).
PCAP — Capacidad de programadores	Bajo	1.15	Errores de compilación básicos sin resolver por días: merges sin cerrar, DI duplicado, argumento repetido en MaterialApp.
TOOL — Uso de herramientas	Alto	0.90	Tracker propio (AppScript) para dailies/tareas, más asistencia de IA para refactors — reduce esfuerzo respecto a un flujo manual.
SCED — Compresión del cronograma	Bajo	1.14	Sprint académico de calendario fijo, comprimido respecto al alcance real construido.
SITE — Desarrollo multi-sitio	Bajo	1.09	Equipo 100% remoto coordinado por WhatsApp + tracker propio, sin infraestructura de colaboración "wideband".

Cálculo de PEM

$$\Pi E_{Mi} = 1.10 \times 1.17 \times 1.14 \times 1.12 \times 1.19 \times 1.15 \times 0.90 \times 1.14 \times 1.09$$
$$\Pi E_{Mi} \approx 2.5149$$

PARTE 4

Esfuerzo, cronograma, personal y presupuesto

Constantes del modelo Post-Architecture: **A = 2.94**, **B = 0.91**, **C = 3.67**, **D = 0.28**.

4.1 Esfuerzo (PM)

$$PM = A \times (\text{Size})^E \times \Pi E_{Mi}$$
$$PM = 2.94 \times (37)^{1.1426} \times 2.5149$$
$$PM = 2.94 \times 61.92 \times 2.5149$$
$$PM \approx 457.8 \text{ persona-mes}$$

4.2 Exponente de cronograma (F)

$$F = D + 0.2 \times (E - B)$$
$$F = 0.28 + 0.2 \times (1.1426 - 0.91)$$
$$F = 0.28 + 0.2 \times 0.2326$$
$$F \approx 0.3265$$

4.3 Tiempo de desarrollo (TDEV)

$$TDEV = C \times (PM)^F$$
$$TDEV = 3.67 \times (457.8)^{0.3265}$$
$$TDEV \approx 3.67 \times 7.393$$
$$TDEV \approx 27.1 \text{ meses}$$

4.4 Personal promedio

$$\text{Personal} = PM / TDEV = 457.8 / 27.1 \approx 16.9 \approx 17 \text{ personas}$$

4.5 Presupuesto estimado

Costo por persona-mes (**aproximado**): se usa **Bs 5,000/persona-mes**, una aproximación razonable para un equipo mixto junior/estudiante en Bolivia — no hay

dato real de nómina porque el equipo no es remunerado.

Presupuesto = $PM \times \text{costo_persona_mes}$

Presupuesto = $457.8 \times 5,000 \text{ Bs}$

Presupuesto $\approx$ **Bs 2,289,100**

458

persona-mes (PM)

0.33

exponente F

27.1

meses (TDEV)

17

personal promedio

Bs 2.29M

presupuesto estimado

PARTE 5

Análisis y reflexión

COCOMO II predice que un sistema del tamaño real de Mi Ruta (37 KSLOC medidos, no estimados) requeriría **≈458 persona-mes**, **≈27 meses** de calendario y un equipo promedio de **≈17 personas** trabajando de forma sostenida y profesional. La comparación con lo que realmente pasó es reveladora, no solo académica: el tracker de seguimiento del propio equipo (docs/AsyncDaily/Revision QA.xlsx) muestra que Sprint 3 —donde se construyó la mayor parte de este código— corrió entre el 1 y el 24 de agosto, es decir, **menos de un mes de calendario**, con 27 personas registradas trabajando en paralelo, a tiempo parcial (turno noche, estudiantes) y sin remuneración. En vez de extender el cronograma hacia los ~27 meses que sugiere COCOMO, el equipo comprimió el tiempo agregando gente en paralelo por encima de cualquier curva de incorporación razonable —el número de ramas simultáneas (~25) superó incluso el personal promedio que el propio modelo calcula como sostenible (17)—. Esa es, casi con seguridad, la causa directa de los problemas que documentamos en este mismo proyecto: dos implementaciones completas de Tickeador construidas en paralelo sin saber una de la otra, dos paneles de Presidente compitiendo, tres tablas de ruteo por rol contradictorias entre sí, y hasta un archivo con credenciales reales commiteado por accidente. COCOMO II lo anticipa indirectamente: los factores de escala más bajos que elegimos (RESL, TEAM, PMAT) son exactamente los que penalizan un equipo grande, mal coordinado y con procesos débiles — y el resultado real coincide con lo que esos factores predicen que puede salir mal, aunque el modelo no capture explícitamente el costo de "reconstruir dos veces la misma funcionalidad".

El **factor de mayor impacto potencial** es **PMAT** (madurez de proceso): tiene el rango más amplio de los 5 factores de escala (7.80 puntos, de Muy Bajo a Extra Alto), más que RESL (7.07) o PREC (6.20), y actúa multiplicando el esfuerzo de forma exponencial a través de E, no linealmente. A nivel de multiplicadores de esfuerzo, **CPLX** tiene el rango

más amplio (0.73–1.74, un factor de 2.4× entre el mejor y el peor caso), coherente con que la complejidad real del planificador de viajes y el modelo de 5 roles es, objetivamente, la parte más difícil de construir bien del sistema.

La limitación más honesta de este ejercicio es que se aplicó **después** de escribir el código, no antes: se midió el tamaño real en vez de proyectarlo, así que esto funciona como una calibración retrospectiva del modelo contra la realidad, no como una predicción a ciegas. COCOMO II tampoco modela bien un equipo voluntario, part-time y radicalmente sobre-paralelizado como este — sus constantes están calibradas contra proyectos industriales con ingenieros de tiempo completo, así que el presupuesto de ~Bs 2.29 millones no representa lo que este proyecto costó realmente (cero, en términos de nómina), sino lo que costaría construirlo de forma profesional y disciplinada. Frente a una estimación intuitiva —contar horas reportadas por persona en los dailies, que darían un número mucho menor, quizás 10-20 persona-mes reales—, COCOMO II aporta lo que la intuición no puede: una cifra objetiva, reproducible y comparable con otros proyectos, aunque a costa de asumir un contexto de ejecución que este proyecto, al ser académico y voluntario, no tuvo.

≈430 palabras

Fuentes: lib/**/*.dart (conteo real de SLOC), git log --merges, docs/AsyncDaily/Revision QA.xlsx (DAILY_SCRUM, TAREAS, OBSERVACIONES), docs/INFORME_AVANCE_SPRINT.md. Tablas de factores de escala y multiplicadores de esfuerzo: material del curso (UT3 - Costos de Software.pdf), COCOMO II Post-Architecture estándar.

Firma Docente Tecnólogo

Firma Director de Carrera

Firma Gestor DESIGN-LAB

Firma CAT KLT